# Governing Human–AI Collaborative Project Workflows in Multi-Platform Digital Delivery: A Design Science Study of Auditable Handoffs and Idempotent Release Operations

**HuaiCheng Sheng**[1,*]

[1]Agentic Work Lab, Hangzhou Chengxin Holding Group Co., Ltd., Hangzhou, Zhejiang, China

[*]Corresponding author. E-mail: `ZackSheng@gmail.com` | ORCID: `0009-0003-2900-6377` | GitHub: `https://github.com/whitecarp`

## ABSTRACT

**Context:** As Large Language Model (LLM) agents transition from conversational assistants to autonomous execution actors in digital engineering and media production projects, project management faces a critical governance gap. Multi-agent workflows frequently suffer from uncoordinated role handoffs, non-idempotent external side effects (such as duplicate platform publications following network timeouts), silent quality degradation, and confounded performance attribution.

**Objective:** Grounded in the Design Science Research (DSR) paradigm and stage-gate project control theory, this study designs, implements, and empirically evaluates an auditable human-in-the-loop (HITL) project governance framework and release operations system (*Agentic Work Lab* and *PublishOps CrossCast*) for multi-agent, multi-platform digital delivery.

**Method:** We formalize a four-layer governance architecture comprising: (1) a deterministic sole-operator and temporal scheduling gate (`planRun`); (2) cryptographic plan-hash locking (`--confirm CURRENT_PLAN_HASH`) coupled with stream-based SHA-256 media integrity verification; (3) an idempotent receipt state machine that latches uncertain post-submission outcomes (`UNKNOWN` states) to suppress automatic retry faults across heterogeneous API and browser-automated endpoints; and (4) an independent read-back verification and HITL exception handoff protocol. The framework is evaluated using real-world longitudinal project artifacts spanning $N = 69$ de-identified operational event logs, $N = 55$ channel publication records, $N = 30$ cryptographically verified video delivery versions (2.21 GB), and 2,737 automated verification test cases across 126 test modules.

**Results:** Across 69 operational checkpoints involving multi-agent coordination (GPT/Codex, Claude, and DeepSeek under human project management), the governance gates suppressed unauthorized mutations in 68 non-write intervals while executing exactly one authorized write (0 duplicate upload replays, 0 new risk incidents). A verified autonomous heartbeat run demonstrated a 24.992 s trigger delay and 31.175 s execution duration with zero task errors. Cross-platform release evaluation across five platforms (Bilibili, YouTube, X, Douyin, and Instagram) confirmed that latching `UNKNOWN` states during programmatic read-back timeouts prevented duplicate publication on Douyin while preserving audit trails. Phase-level latency decomposition on Instagram (423.236 s end-to-end; 412.217 s publication command) revealed that 92.32% (380.551 s) of command duration resides in post-submission platform confirmation wait, identifying the primary vulnerability window for non-idempotent agent retries.

**Conclusions:** Effective project management of agentic workflows requires shifting from prompt-level persuasion to deterministic stage-gate invariants, cryptographic artifact locking, and idempotent state-machine recovery. All source code, schemas, and de-identified empirical datasets are publicly archived to establish a reproducible benchmark for agentic project governance.

**Keywords:** Project Management Governance; Human–AI Collaboration; Multi-Agent Workflows; Design Science Research; Idempotent Release Engineering; Stage-Gate Control; PublishOps

**Research Highlights**

- Proposes a four-layer Design Science governance framework for human–AI collaborative project delivery.
- Formalizes an idempotent receipt state machine that prevents duplicate side effects under `UNKNOWN` network/UI states.
- Evaluates multi-agent handoffs (GPT/Codex, Claude, DeepSeek) across 69 operational events and 30 SHA-256 verified artifacts.
- Demonstrates across 5 platforms that 92.32% of browser-mediated release latency occurs in post-submission confirmation waits.
- Establishes epistemic audit rules separating descriptive cohort metrics ($N = 55$) from confounded causal claims.

## 1. Introduction

The rapid maturation of Large Language Models (LLMs) and autonomous tool-using agents has initiated a structural transformation in project management and systems engineering (Holzmann et al., 2022; Müller et al., 2024; Wang et al., 2024). Unlike traditional software scripts that execute fixed, deterministic subroutines, agentic workflows deploy heterogeneous foundation models—such as reasoning-oriented coding agents, interface construction agents, and scheduled monitoring agents—to collaborate on complex, multi-stage deliverables (Wu et al., 2024; Hong et al., 2024). In digital content production, software release engineering, and routine operations, these multi-agent topologies promise substantial reductions in manual coordination overhead (Raisch & Krakowski, 2021; Berente et al., 2021).

However, deploying autonomous or semi-autonomous AI agents into real-world production environments exposes severe project governance vulnerabilities that classical project management frameworks were not designed to address (PMI, 2021; Kerzner, 2022). From a project control perspective, three fundamental failure modes emerge when LLM agents interact with external production platforms:

1. **Multi-Agent Handoff Drift and Role Ambiguity:** When multiple specialized models (e.g., GPT/Codex for script synthesis and backend verification, Claude for interface construction, and DeepSeek for scheduled operational monitoring) operate asynchronously on shared project state, context boundaries frequently degrade. Without formal ownership locks and explicit acceptance gates, an agent may re-execute a task already claimed by another instance, overwrite human-approved revisions, or act on stale schedule checklists (Whyte et al., 2016; Qian et al., 2024).
2. **Non-Idempotent External Side Effects under Uncertainty:** Real-world release operations across heterogeneous platforms (such as Bilibili, YouTube, X, Douyin, and Instagram) rely on a mixture of official REST APIs and stateful browser sessions. Network interruptions, platform review queues, CAPTCHA challenges, or DOM read-back timeouts frequently leave a submission in an indeterminate (`UNKNOWN`) state (Schneider, 1990; Kleppmann, 2017). Naive agentic retry loops interpret a missing confirmation receipt as a failed upload and automatically re-trigger submission, resulting in duplicate public posts, corrupted metadata, or account rate-limiting penalties.
3. **Epistemic Confounding in Performance Evaluation:** In industry and academic reports alike, the introduction of AI workflows is frequently accompanied by unverified claims of productivity speedup or audience growth (Raisch & Krakowski, 2021). When comparing historical human-led project outputs against newly launched AI-assisted series, differences in artifact exposure age, topic selection, and platform recommendation dynamics confound naive metric aggregation. Furthermore, treating generated draft volume as equivalent to verified, production-ready delivery obscures the substantial human review and rework required to achieve release acceptance.

To address these challenges, this study adopts the Design Science Research (DSR) methodology (Hevner et al., 2004; Peffers et al., 2007; Gregor & Hevner, 2013) to conceptualize, build, and evaluate a comprehensive human-in-the-loop (HITL) project governance framework and release operations console: **Agentic Work Lab** and its core release engine **PublishOps CrossCast** (`vpub`). Rather than treating AI governance as an informal prompt-engineering exercise, we formulate multi-agent project delivery as a verifiable stage-gate control system (Cooper, 1990; Bianchi et al., 2022) backed by cryptographic artifact locking and fault-tolerant state-machine transitions.

Specifically, this paper investigates the following three research questions (RQs):

- **RQ1 (Governance Architecture):** How can stage-gate project management principles be formalized into deterministic software invariants to govern multi-model handoffs and prevent unauthorized or duplicate autonomous actions?
- **RQ2 (Fault-Tolerant Release Engineering):** How can an idempotent receipt state machine coordinate multi-platform digital releases across hybrid API and browser-automated channels when post-submission verification encounters timeouts or human authentication barriers?
- **RQ3 (Empirical & Epistemic Validation):** What are the empirical operational characteristics, latency bottlenecks, and methodological boundaries observed when deploying this governance architecture across longitudinal multi-platform production workflows?

The primary contributions of this work are fourfold: (1) formalizing a four-layer agentic project governance model bridging stage-gate control and human-centered automation; (2) presenting the open-source reference architecture of *PublishOps CrossCast* (`v0.16.0` and main branch through PR #30, comprising 123 Python source modules / 27,095 LOC and 126 test modules / 37,155 LOC across 2,737 test functions); (3) conducting longitudinal empirical trace and phase-latency decomposition across 69 operational events, 30 SHA-256 verified video versions (2.21 GB), and 5 publishing platforms; and (4) establishing a machine-verifiable epistemic reporting protocol that prevents survivorship bias and exposure-age confounding across $N = 55$ channel records.

## 2. Related Work and Theoretical Background

### *2.1 Project Management Governance and Stage-Gate Control in the AI Era*

Classical project management theory emphasizes structured control mechanisms—such as Work Breakdown Structures (WBS), configuration management, and Stage-Gate reviews—to manage uncertainty and enforce accountability across project phases (Cooper, 1990; Whyte et al., 2016; PMI, 2021; Kerzner, 2022). In hybrid agile–stage-gate models (Bianchi et al., 2022), iterative execution within a sprint is bounded by formal quality gates that verify deliverable completeness before authorizing downstream commitment.

Recent scholarship in project management has begun examining how artificial intelligence alters project governance, decision-making authority, and human–machine agency (Holzmann et al., 2022; Müller et al., 2024). Raisch & Krakowski (2021) highlight the twin perils of over-automation (where removing human judgment leads to systemic drift and brittle failure) and isolated augmentation, arguing that organizations must design explicit coupling protocols between human managers and algorithmic actors. Similarly, Parasuraman et al. (2000) and Shneiderman (2020) emphasize that high-reliability systems require combining high levels of automation with high levels of human control through transparent audit trails, explicit pre-authorization, and graceful handoff under exception states. Despite these theoretical advances, existing project management literature largely relies on conceptual typologies or Delphi surveys (Holzmann et al., 2022); there remains a scarcity of concrete, software-enforced governance architectures that operationalize stage-gate controls for live, multi-model AI agents.

### *2.2 Multi-Agent LLM Workflows and Coordination Failures*

The emergence of tool-augmented LLM agents (Yao et al., 2023; Shinn et al., 2023) has spurred the development of multi-agent collaborative frameworks such as AutoGen (Wu et al., 2024), MetaGPT (Hong et al., 2024), and ChatDev (Qian et al., 2024). By decomposing complex workflows into specialized agent roles and employing Standardized Operating Procedures (SOPs), these frameworks reduce single-context hallucination on synthetic coding benchmarks (Wang et al., 2024). Nevertheless, empirical deployments in continuous operations reveal persistent coordination vulnerabilities: conversational handoffs lack transactional atomicity, and production workflows interact with stateful external platforms where unauthorized writes, duplicate submissions, or unintended deletions are irreversible (Amershi et al., 2019). Governing heterogeneous models in production therefore requires externalizing state management from volatile LLM context windows into deterministic file and lock structures.

### *2.3 DevOps, Release Engineering, and Idempotent State Machines*

In software engineering, Continuous Integration and Continuous Delivery (CI/CD) pipelines manage deployment risk through automated build verification, immutable artifact digests, and staged rollouts (Humble & Farley, 2010; Bass et al., 2015; Adams & McIntosh, 2016; Shahin et al., 2017; Leite et al., 2019). A foundational principle of reliable distributed systems is *idempotency*—the guarantee that repeating an operation or recovering from an interrupted request does not produce duplicate side effects (Schneider, 1990; Gray & Reuter, 1992; Kleppmann, 2017). Extending DevOps principles to multi-platform digital media delivery (*PublishOps*) introduces unique socio-technical constraints across official OAuth REST APIs and stateful web browser interfaces, necessitating a unified state-machine governance layer.

## 3. Research Methodology

### *3.1 Design Science Research (DSR) Framework*

We structure this investigation following the six-step Design Science Research Methodology (DSRM) established by Peffers et al. (2007) and the evaluation principles of Hevner et al. (2004) and Venable et al. (2016): (1) Problem Identification and Motivation; (2) Definition of Solution Objectives; (3) Design and Development of the four-layer governance architecture and state-machine invariants; (4) Longitudinal Demonstration across two active production series (*GPT Duty Log* and *Whale Girl Shift Log*) and standalone short films across five platforms; (5) Multi-Method Evaluation combining operational trace analysis ($N = 69$), software fault-injection testing (2,747 total tests), cross-platform release timing, and cohort analytics ($N = 55$); and (6) Public Archival Communication.

### *3.2 Epistemic Hygiene and Evidence Classification Protocol*

Every operational record and research note in the laboratory is governed by four mandatory reporting rules (Runeson & Höst, 2009): (1) strict separation of primary observational receipts from causal inferences; (2) explicit state granularity distinguishing preparation, rehearsal, preflight authentication, submission wait, platform review, and public visibility (recording timed-out verifications explicitly as `UNKNOWN`); (3) immutable snapshot preservation across sampling dates; and (4) explicit documentation of unexecuted test plans and measurement boundaries.

## 4. The Four-Layer Human–AI Project Governance Architecture

To answer **RQ1** and **RQ2**, we formalize the project governance architecture as a four-layer hierarchical control system mediating between the human Project Manager, specialized LLM agents, local media/code artifacts, and external publishing platforms (Figure 1).

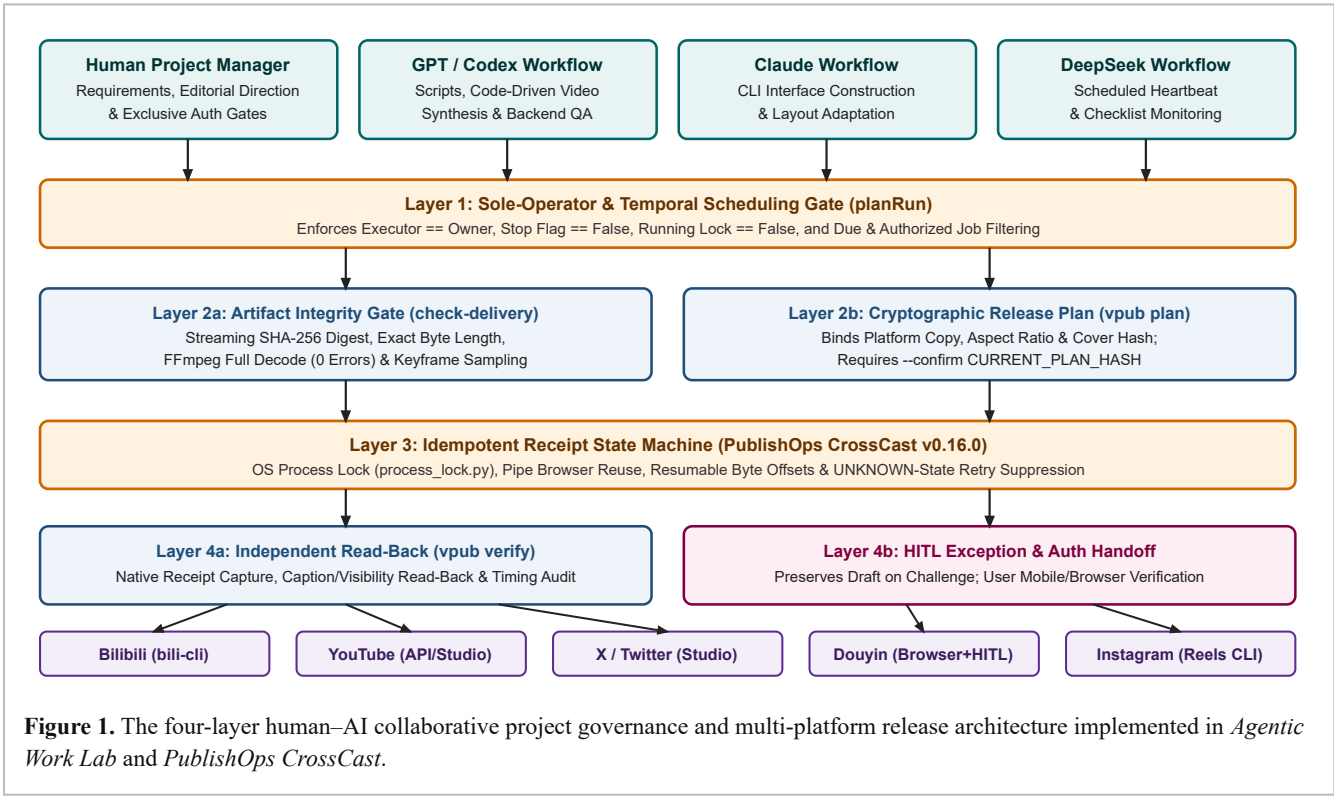



**Figure 1.** The four-layer human–AI collaborative project governance and multi-platform release architecture implemented in *Agentic Work Lab* and *PublishOps CrossCast*.

### *4.1 Multi-Agent Role Allocation and Handoff Boundaries*

Within the laboratory environment, project execution is partitioned across four distinct actors: the Human Project Manager and three specialized AI agent workflows (GPT/Codex, Claude, and DeepSeek). All inter-agent handoffs are mediated through version-controlled state files (`operations-checklist.md`, `delivery-manifest.json`, and

`production-update-*.json`). Table 1 defines the task scope, handoff artifacts, and operational constraints for each actor.

**Table 1: Multi-agent role allocation, handoff artifacts, and governance boundaries across the project lifecycle.**

| Project Actor | Assigned Functional Scope | Observable Evidence & Handoff Artifacts | Enforced Governance & Safety Constraints |
|---|---|---|---|
| **Human Project Manager** (Sole Operator / Architect) | Requirements definition, editorial direction, creative acceptance, platform authentication, and final release authorization. | Explicit authorization events (`USER_AUTHORIZED_*`), mobile/browser verification receipts, and plan confirmation hashes. | Retains exclusive sign-off on public releases, deletions, and schedule overrides; resolves `UNKNOWN` platform states. |
| **GPT / Codex Workflow** (Production & Backend QA) | Topic/merchant research, script synthesis, programmable video timeline assembly, FFmpeg encoding checks, and backend repository review. | SHA-256 verified `.mp4` exports, frame-sampling logs, decode error counts, and pull request verification records. | Cannot self-authorize public platform writes; unapproved or in-progress renders (`RENDER_OR_QA_PENDING`) are excluded from release. |
| **Claude Workflow** (Interface & Tool Construction) | CLI interface construction, layout adaptation, portrait cut tooling, and developer user-experience refinements in *CrossCast*. | Scoped pull requests (e.g., PR 8 capability list, PR 9 first-run checks, PR 10–30 workflows) and unit test suites. | Changes remain commit-scoped until verified by regression suites; local mock tests are never reported as live speedups. |
| **DeepSeek Workflow** (Scheduled Operations Monitor) | Hourly automated heartbeat inspections, reading operational checklists, checking due times, and synchronizing user instructions. | Machine-readable execution traces (`scheduled-run-evidence.json`) and de-identified event logs (`operations-events.json`). | Strictly read-only when no authorized task is due; forbidden from launching duplicate runs or retrying uncertain submissions. |

### *4.2 Layer 1: Deterministic Sole-Operator and Temporal Scheduling Gate*

To prevent race conditions and unauthorized autonomous execution during scheduled heartbeats, Layer 1 enforces a pure offline decision function $G_{\text{plan}}$ before any network or platform adapter is invoked. Let $S = (e, o, f_{\text{stop}}, f_{\text{run}}, t_{\text{now}}, J)$ denote the operational context tuple, where $e$ is the requesting executor identifier, $o$ is the registered instance owner, $f_{\text{stop}} \in \{\text{true}, \text{false}\}$ is the emergency halt flag, $f_{\text{run}} \in \{\text{true}, \text{false}\}$ indicates whether an execution lock is active, $t_{\text{now}} \in \mathbb{R}^{+}$ is the current epoch timestamp, and $J = \{j_1, \ldots, j_m\}$ is the set of candidate jobs.

$$G_{\text{plan}}(S) = [\ (\texttt{skip}, \texttt{not_owner}, \emptyset) \text{ if } e = \emptyset \vee e \neq o;\quad (\texttt{skip}, \texttt{stop_flag}, \emptyset) \text{ if } f_{\text{stop}} = \text{true};\quad (\texttt{skip}, \texttt{already_running}, \emptyset) \text{ if } f_{\text{run}} = \text{true};\quad (\texttt{plan_only}, \texttt{checked}, D) \text{ if } D \neq \emptyset;\quad (\texttt{no_work_due}, \texttt{checked}, \emptyset) \text{ otherwise }] \tag{1}$$

where the due set $D$ is strictly filtered as:

$$D = \{\, id_k \mid j_k \in J \wedge done_k = \text{false} \wedge auth_k = \text{true} \wedge t_{due,k} \leq t_{now} \,\} \tag{2}$$

Furthermore, for any automated run record $R = (\text{trig}, \text{status}, f_{\text{manual}}, t_{\text{start}}, t_{\text{end}}, E_{\text{err}})$, post-execution verification $V_{\text{sched}}(R)$ holds if and only if:

$$V_{\text{sched}}(R) \Leftrightarrow (\text{trig} = \texttt{automation_heartbeat_scheduled}) \wedge (\text{status} = \texttt{VERIFIED_AUTOMATIC_RUN_COMPLETED}) \wedge (f_{\text{manual}} = \text{false}) \wedge (0 < t_{\text{start}} \leq t_{\text{end}} < \infty) \wedge (|E_{\text{err}}| = 0) \tag{3}$$

### *4.3 Layer 2: Cryptographic Plan-Hash Locking and Artifact Verification*

**Governance Invariant 1 (Binary Media Integrity):** Let $M = \{m_1, \ldots, m_K\}$ be the release manifest where each entry $m_i = (\text{fname}_i, B_i, H_i)$ specifies a basename-sanitized file, exact byte length $B_i$, and expected SHA-256 digest $H_i$. An artifact $f_i$ passes verification iff $\text{size}(f_i) = B_i \wedge \text{SHA256}(f_i) = H_i \wedge \text{DecodeErrors}(f_i) = 0$.

**Governance Invariant 2 (Plan-Hash Confirmation Binding):** Before invoking any platform write via `vpub publish`, a canonical release plan $P$ is hashed as $H(P) = \text{SHA256}(\text{CanonicalJSON}(P))$. Execution is rejected unless the command supplies `--confirm` $H_{\text{user}}$ such that $H_{\text{user}} = H(P)$.

### *4.4 Layer 3: Idempotent Receipt State Machine and UNKNOWN-State Suppression*

To answer **RQ2**, we model cross-platform publication in *PublishOps CrossCast* as a persistent state machine backed by cross-process OS locks (`process_lock.py`) and durable local receipt journals (Figure 2).

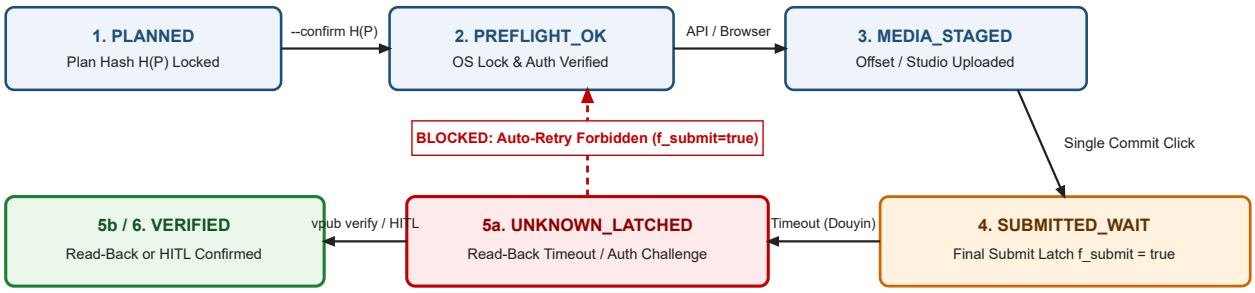



**Figure 2.** The idempotent receipt state machine in *PublishOps CrossCast*. Once the final-submit latch (`f_submit = true`) is persisted at State 4, any subsequent timeout transitions to `UNKNOWN_LATCHED` (State 5a), deterministically blocking automatic retries.

## 5. System Implementation: Agentic Work Lab and PublishOps CrossCast

### *5.1 Programmable Video Synthesis and Multi-Modal Quality Gates*

Within *Agentic Work Lab*, video production is engineered as an editable, code-driven workflow (`editable-video-workflow.md`) rather than opaque one-shot video generation. Audio narration timelines, pose transitions, camera movements, and bilingual subtitles are defined programmatically so individual defects (such as flash frames in *GPT Duty Log 05* or heel-sound synchronization in *Whale Girl* studies) can be patched deterministically. Every deliverable undergoes streaming SHA-256 verification (`check-delivery.mjs`), full FFmpeg container decoding (`full_decode_errors = 0`), and keyframe sampling (e.g., 37 sampled frames for *I Watched It For You v3*).

### *5.2 CrossCast Multi-Platform Subsystem Architecture*

*PublishOps CrossCast* (public release `v0.16.0`, commit `a46bd19`; main-branch evolution through commit `75b7abc`, PR #30) is implemented in Python across 123 source modules (27,095 LOC) and validated by 126 test modules (37,155 LOC, 2,737 test functions). Following the `v0.16.0` baseline snapshot (`69aa590`), subsequent main-branch engineering (PRs #10–#30) delivered the two-command `prepare`/`go` release workflow, bounded two-slot concurrent browser publication (`066c49f`, `6f7b62b`), explicit post-submission confirmation wait reporting (`c33dcfc`), and durable-consent single-work deletion with absence read-back verification (`3b56bdc`, `75b7abc`).

## 6. Empirical Evaluation and Results

### *6.1 Operational Event Trace Analysis and Write Suppression (RQ1)*

Table 2 summarizes the taxonomy and governance metrics across the complete de-identified operational event journal (`data/operations-events.json`, $N = 69$ events) alongside the verified autonomous heartbeat trace (`data/scheduled-run-evidence.json`).

**Table 2: Taxonomy and governance metrics across $N = 69$ de-identified operational event records in *Agentic Work Lab*.**

| Operational Event Category | Events | Share | Reads | Writes | Primary Governance Mechanism Verified |
|---|---|---|---|---|---|
| Scheduled Zero-Write Heartbeat & Idle Checks | 32 | 46.38% | 9 | 0 | Layer 1 (`planRun`): Suppresses writes when no task is due (`no_work_due`). |
| Human Governance, Topic & Release Auth Gates | 16 | 23.19% | 4 | 0 | Layer 1 & 4: Explicit human sign-off (`USER_AUTHORIZED_*`) & preview reply gates. |
| Production Delivery, Temporal QA & Revisions | 16 | 23.19% | 4 | 0 | Layer 2: Frame/decode checks, visual revision loops & spare preservation. |
| Station Sampling & Trend Research Queries | 4 | 5.80% | 0 (26 CLI) | 0 | Bounded read-only CLI sampling (`cli_calls = 26`) without platform writes. |
| Authorized Platform Write Execution | 1 | 1.45% | 4 | 1 | Controlled single-write execution with read verification (`new_risk = false`). |
| **Total / Aggregate Audit Metrics** | **69** | **100.00%** | **17** | **1** | **Duplicate Replays: 0 \| New Risk Events: 0** |

Out of 69 operational events, 68 (98.55%) executed with zero platform writes, zero duplicate upload replays, and zero new risk incidents. In the audited DeepSeek scheduled run (`2026-10-28T12:37:13.943+08:00`), the heartbeat triggered after a scheduling delay of 24,992 ms (24.992 s) and completed its four-step inspection in 31,175 ms (31.175 s) with zero task errors and no duplicate trigger.

### *6.2 Software Verification and Fault-Injection Test Coverage (RQ1 & RQ2)*

**Table 3: Subsystem code volume and automated fault-injection verification coverage across *PublishOps CrossCast* and *Agentic Work Lab*.**

| Subsystem / Verification Category | Files | Test LOC | Test Cases | Key Fault Modes & Invariants Tested |
|---|---|---|---|---|
| ***A. PublishOps CrossCast (v0.16.0 / main 75b7abc: 123 source files, 27,095 source LOC)*** | | | | |
| 1. Workflow, CLI, Scheduling & Notifications | 66 | 17,020 | 1,266 | Two-command `prepare` / `go` gates, plan-hash validation, two-slot concurrency, Gmail self-send claims. |
| 2. Platform Adapters & Publishing (Bili, YT, X, DY, IG) | 18 | 7,016 | 562 | Aspect-ratio guards, FFmpeg cover lead-in synthesis, X post length checks, YouTube Studio zones. |
| 3. Deletions, Edits & Read-Back Verification | 24 | 8,575 | 485 | `expected_identity` binding, `EDIT_MISMATCH` recovery, durable-consent single-work deletion & absence read-back. |
| 4. Collections, Playlists & Catalog Discovery | 10 | 3,577 | 322 | Account-bound collection IDs, partial-page scroll guards, lost-response recovery, playlist deduplication. |
| 5. Idempotency, Receipts, Locks & Recovery | 8 | 967 | 102 | OS process lock recovery after confirmed exit, resumable upload byte offsets, `UNKNOWN` retry blocking. |
| **Subtotal (PublishOps CrossCast)** | **126** | **37,155** | **2,737** | **Test-to-Source LOC Ratio: 1.37 : 1** |
| ***B. Agentic Work Lab Reproducible Entry Points (Node.js v26.11.1)*** | | | | |
| 6. Workflow Gate & Merchant Demo (`workflows.test.mjs`) | 1 | 38 | 10 | Non-owner rejection, stop-flag blocking, running-task lock, invalid due-time exception (10/10 passed in 108.5 ms). |
| 7. Archive Cryptographic Integrity (`manifest.json`) | 1 | — | 30 | Stream SHA-256 & exact byte checks across 30 downloadable video versions (2,211,716,129 bytes; 0 mismatches). |

### *6.3 Cross-Platform Release Verification and Latency Bottleneck Analysis (RQ2 & RQ3)*

**Table 4: Cross-platform publication verification matrix for *I Watched It For You (v3)* (176.4 s, 285,855,005 bytes, SHA-256 `7d6fd760...`).**

| Platform | Transport Route | Native Receipt State | Verification State | Observed Governance Outcome |
|---|---|---|---|---|
| **Bilibili** | `bili-cli` Session Reuse | `REPORTED_PUBLIC` | `PUBLIC_PER_SESSION` (`BV15B4F6hEYr`) | Public availability recorded at 2 Nov 21:00 UTC+8; state preserved without redundant re-query. |
| **YouTube** | Official API / Studio Draft Continuation | `SUBMISSION_VERIFIED` | `PUBLIC_VERIFIED` (`AzW6vr8nvDM`) | Continued from original draft; native receipt, bilingual copy, and visibility passed automated read-back. |
| **X (Twitter)** | Official Web / Media Studio Asset Reuse | `SUBMISSION_VERIFIED` | `PUBLIC_VERIFIED` (`2117261340633161838`) | Reused original uploaded media; receipt and 176.4 s duration verified; stored resolution flagged unestablished. |
| **Douyin** | CLI Background Browser + HITL | **`UNKNOWN`** (Read-Back Timed Out) | `USER_CONFIRMED_VISIBLE` | **Idempotency Validation:** Timeout latched `UNKNOWN`; blocked automatic retry; operator phone confirmed live work. |
| **Instagram** | CLI Background Browser (Reels) | `SUBMISSION_VERIFIED` | `PUBLIC_VERIFIED` (`DfQ9iCiCWYN`) | Single upload and single Share; native receipt, ZH/EN caption, 16:9 ratio, and AI label verified via read-back. |

**Table 5: Phase-level latency decomposition of the live Instagram publication workflow (*N* = 1 instrumented observation).**

| Execution Phase | Execution Scope | Duration (s) | % of Cmd | % of E2E |
|---|---|---|---|---|
| 1. Source & Release Plan Cryptographic Checks | Outside Publish Cmd | 0.438 | — | 0.10% |
| 2. Local Dry-Run Rehearsal | Outside Publish Cmd | 0.444 | — | 0.10% |
| 3. Sign-In Preflight & Session Close | Outside Publish Cmd | 10.137 | — | 2.40% |
| ***Subtotal: Pre-Publication Governance Checks*** | ***Outside Publish Cmd*** | ***11.019*** | **—** | ***2.60%*** |
| 4. Browser Start to Media File Entry | Inside `vpub publish` | 11.058 | 2.68% | 2.61% |
| **5. Post-Submission Wait (Transfer, Processing & Confirmation)** | **Inside `vpub publish`** | **380.551** | **92.32%** | **89.91%** |
| 6. Independent Post-Publication Read-Back | Inside `vpub publish` | 5.224 | 1.27% | 1.23% |
| 7. Residual In-Command Form/UI Overhead | Inside `vpub publish` | 15.384 | 3.73% | 3.63% |
| ***Subtotal: Publication Command Total*** | ***Inside `vpub publish`*** | ***412.217*** | ***100.00%*** | ***97.40%*** |
| **Total End-to-End Observed Workflow Time** | **All Recorded Phases** | **423.236** | **—** | **100.00%** |

### *6.4 Descriptive Cohort Analytics and Epistemic Boundary Control (RQ3)*

**Table 6: Descriptive statistical comparison of the historical human-led cohort (*n* = 45 published) and the AI-assisted series cohort (*n* = 10 total: 5 published, 5 scheduled) at the 28 October 2026 snapshot, alongside 24-hour checkpoints.**

| Cohort / Metric | Mean | SD | Median | IQR (Q1–Q3) | Min–Max | Sum |
|---|---|---|---|---|---|---|
| ***A. Historical Human-Led Cohort (human_led_history: n = 45 published, Oct 2019 – Oct 2026)*** | | | | | | |
| Lifetime Views | 40,884.31 | 64,604.81 | 15,407.0 | 1,722–67,502 | 274–333,725 | 1,839,794 |
| Likes | 4,022.36 | 7,281.57 | 729.0 | 89–3,765 | 8–30,299 | 181,006 |
| Comments | 78.18 | 132.65 | 14.0 | 4–113 | 0–597 | 3,518 |
| Favorites | 1,584.31 | 2,048.27 | 473.0 | 19–2,746 | 2–6,158 | 71,294 |
| Coins | 356.38 | 751.32 | 54.0 | 6–237 | 2–3,044 | 16,037 |
| Shares | 764.76 | 1,594.05 | 31.0 | 1–996 | 0–5,571 | 34,414 |
| Danmaku (Bullet Chats) | 190.27 | 232.37 | 105.0 | 41–279 | 0–1,045 | 8,562 |
| ***B. Current AI-Assisted Series (current_ai_series: n = 5 published, 5 scheduled, 25–28 Oct 2026)*** | | | | | | |
| Early Views (≤ 3 days age) | 133.20 | 99.53 | 183.0 | 42–199 | 11–231 | 666 |
| Likes | 10.60 | 7.70 | 9.0 | 8–11 | 2–23 | 53 |
| Comments | 29.40 | 46.94 | 12.0 | 7–13 | 2–113 | 147 |
| Favorites | 4.20 | 5.26 | 2.0 | 1–5 | 0–13 | 21 |
| ***C. Age-Matched 24-Hour Post-Publication Checkpoints (Single-Work Descriptive Records)*** | | | | | | |
| **GPT Duty Log 03** ( `BV1UXxN6aECf` , 70 s, age 24.38 h): 229 views, 8 likes (3.49%), 7 replies (3.06%), 44.64% completion (31 s avg watch). | | | | | | |
| **Whale Girl 02** ( `BV1Wfx56JEih` , 117 s, age 24.05 h): 308 views, 24 likes (7.79%), 51 replies (16.56%), 36.63% completion (42 s avg watch). | | | | | | |

## 7. Discussion

### *7.1 Theoretical Implications for Project Management and Human–AI Governance*

Our findings contribute to the emerging theory of AI-augmented project management (Holzmann et al., 2022; Müller et al., 2024; Raisch & Krakowski, 2021) in three fundamental ways. First, whereas prior multi-agent frameworks (Wu et al., 2024; Hong et al., 2024; Qian et al., 2024) rely primarily on natural-language role prompts to coordinate task handoffs, our empirical trace shows that safe delegation in continuous operations requires externalizing stage-gate controls into deterministic code invariants ( `planRun` , OS process locks, and `--confirm CURRENT_PLAN_HASH` ). This bridges classical project configuration management (Whyte et al., 2016; Cooper, 1990) with modern agentic systems engineering. Second, in human–automation interaction theory (Parasuraman et al., 2000; Shneiderman, 2020), automation authority must be *state-dependent*: agents can operate with high autonomy during preflight verification and byte staging (States 1–3), but must immediately surrender retry authority to a conservative latch ( `UNKNOWN_LATCHED` , State 5a) once an external commit flag is set. Third, by embedding provenance tracking, SHA-256 artifact manifests, and non-causal limitation tags directly into the reporting pipeline, *Agentic Work Lab* establishes epistemic governance as a first-class project management deliverable.

### *7.2 Managerial and Engineering Lessons Learned*

- **Measure the Wait Before Optimizing Concurrency:** Because 92.32% of live browser-mediated publication time occurs inside platform confirmation waits after submission (Table 5), optimizing local script startup yields negligible end-to-end speedup; engineering effort should focus on safe session isolation and bounded cross-platform concurrency.

- **Design Editable Intermediate Representations:** Decomposing deliverables into code-driven timelines, separate audio tracks, and structured JSON patches allows human project managers to enforce surgical revisions without discarding validated upstream work.
- **Preserve Original State on Verification Interruptions:** When platform security challenges or missing edit permissions occur, tools must never automatically delete, repost, or overwrite existing works.

### *7.3 Threats to Validity and Limitations*

Consistent with software engineering case study guidelines (Runeson & Höst, 2009), we explicitly document the boundaries of this study: (1) *Internal Validity:* The Instagram latency decomposition (412.217 s) and the verified DeepSeek scheduled run (31.175 s) represent exact instrumented single-run observations ($N = 1$ each) rather than multi-trial statistical benchmarks, and client-side measurements cannot separate upload transfer from server-side transcoding during the 380.551 s confirmation wait. (2) *Construct Validity:* The pre-registered blinded identical-task multi-model merchant evaluation (comparing GPT and DeepSeek on identical prompts with token-cost logging) remains an unexecuted plan and is not claimed as an empirical result here. (3) *External Validity:* Empirical records derive from a single creator/engineering laboratory across five platforms (with TikTok integration suspended due to regional authentication constraints).

## 8. Conclusion and Future Work

This paper presented a Design Science study addressing how to govern human–AI collaborative project workflows in multi-platform digital delivery. By architecting and evaluating *Agentic Work Lab* and *PublishOps CrossCast*, we demonstrated that reliable multi-agent operations require combining clear role handoff boundaries (across human project management, GPT/Codex, Claude, and DeepSeek) with a four-layer deterministic governance framework: (1) sole-operator and temporal scheduling gates, (2) cryptographic plan-hash and media SHA-256 verification, (3) an idempotent receipt state machine that latches `UNKNOWN` post-submission states to eliminate duplicate retries, and (4) independent read-back and human-in-the-loop verification handoffs. Empirical evidence across 69 operational events, 30 SHA-256 verified video versions (2.21 GB), 2,747 automated tests, and five live publishing platforms confirms that the architecture prevents unauthorized writes and duplicate publications while maintaining a transparent, reproducible audit trail.

## Data and Code Availability Statement

All source code, workflow verification scripts, de-identified operational event logs ($N = 69$), channel cohort snapshots ($N = 55$), and SHA-256 verified media release manifests ($N = 30$) are openly available at `https://github.com/whitecarp/agentic-work-lab` and `https://github.com/whitecarp/publishops-crosscast`.

## Declaration of Competing Interest

The author declares no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## References


1. Adams, B., & McIntosh, S. (2016). Modern release engineering in a nutshell—Why researchers should care. *IEEE SANER*, 5, 78–90.
2. Amershi, S., Weld, D., Vorvoreanu, M., et al. (2019). Guidelines for human-AI interaction. *Proceedings of ACM CHI*, 1–13.
3. Bass, L., Weber, I., & Zhu, L. (2015). *DevOps: A Software Architect's Perspective*. Addison-Wesley Professional.
4. Berente, N., Gu, B., Recker, J., & Santhanam, R. (2021). Managing artificial intelligence. *MIS Quarterly*, 45(3), 1433–1450.
5. Bianchi, M. J., Conforto, E. C., Rebentisch, E., et al. (2022). Stage-Gate and agile project management: A framework for hybrid models. *IEEE Transactions on Engineering Management*, 69(6), 3590–3603.
6. Cooper, R. G. (1990). Stage-Gate systems: A new tool for managing new products. *Business Horizons*, 33(3), 44–54.
7. Gray, J., & Reuter, A. (1992). *Transaction Processing: Concepts and Techniques*. Morgan Kaufmann.

8. Gregor, S., & Hevner, A. R. (2013). Positioning and presenting design science research for maximum impact. *MIS Quarterly*, 37(2), 337–355.
9. Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Quarterly*, 28(1), 75–105.
10. Holzmann, V., Zitter, D., & Peshkess, S. (2022). The expectations of project managers from artificial intelligence: A Delphi study. *Project Management Journal*, 53(5), 438–455.
11. Hong, S., Zhuge, M., Chen, J., et al. (2024). MetaGPT: Meta programming for a multi-agent collaborative framework. *ICLR*.
12. Humble, J., & Farley, D. (2010). *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley.
13. Kerzner, H. (2022). *Project Management: A Systems Approach to Planning, Scheduling, and Controlling* (13th ed.). Wiley.
14. Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.
15. Leite, L., Rocha, C., Kon, F., Milojicic, D., & Meirelles, P. (2019). A survey of DevOps concepts and challenges. *ACM Computing Surveys*, 52(6), 1–35.
16. Müller, R., Locatelli, G., & Holzmann, V. (2024). Artificial intelligence and project management: Governance, agency, and human-AI collaboration. *International Journal of Project Management*, 42(1), 102554.
17. Parasuraman, R., Sheridan, T. B., & Wickens, C. D. (2000). A model for types and levels of human interaction with automation. *IEEE Transactions on Systems, Man, and Cybernetics - Part A*, 30(3), 286–297.
18. Peffers, K., Tuunanen, T., Rothenberger, M. A., & Chatterjee, S. (2007). A design science research methodology for information systems research. *Journal of Management Information Systems*, 24(3), 45–77.
19. Project Management Institute (PMI). (2021). *A Guide to the Project Management Body of Knowledge (PMBOK Guide)* (7th ed.). PMI.
20. Qian, C., Liu, W., Liu, H., et al. (2024). ChatDev: Communicative agents for software development. *Proceedings of ACL*, 15174–15186.
21. Raisch, S., & Krakowski, S. (2021). Artificial intelligence and management: The automation–augmentation paradox. *Academy of Management Review*, 46(1), 192–210.
22. Runeson, P., & Höst, M. (2009). Guidelines for conducting and reporting case study research in software engineering. *Empirical Software Engineering*, 14(2), 131–164.
23. Schneider, F. B. (1990). Implementing fault-tolerant services using the state machine approach: A tutorial. *ACM Computing Surveys*, 22(4), 299–319.
24. Shahin, M., Babar, M. A., & Zhu, L. (2017). Continuous integration, delivery and deployment: A systematic review. *IEEE Access*, 5, 3909–3943.
25. Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., & Yao, S. (2023). Reflexion: Language agents with verbal reinforcement learning. *NeurIPS*, 36, 8634–8652.
26. Shneiderman, B. (2020). Human-centered artificial intelligence: Reliable, safe & trustworthy. *International Journal of Human–Computer Interaction*, 36(6), 495–504.
27. Venable, J., Pries-Heje, J., & Baskerville, R. (2016). FEDS: A framework for evaluation in design science research. *European Journal of Information Systems*, 25(1), 77–89.
28. Wang, L., Ma, C., Feng, X., et al. (2024). A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6), 186345.
29. Whyte, J., Stasis, A., & Lindkvist, C. (2016). Managing change in the delivery of complex projects: Configuration management, asset information and 'big data'. *International Journal of Project Management*, 34(2), 339–351.
30. Wu, Q., Bansal, G., Zhang, J., et al. (2024). AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *COLM*.
31. Yao, S., Zhao, J., Yu, D., et al. (2023). ReAct: Synergizing reasoning and acting in language models. *ICLR*.